

# PandOSsh — Fase 1

Le strutture dati di base: liste, PCB e ASL  
Guida dettagliata riga per riga, funzione per funzione

Materiale di studio per l'interrogazione

A.A. 2025/2026

## Indice

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione generale alla Fase 1</b>                                | <b>2</b>  |
| <b>2</b> | <b>headers/types.h — i tipi di dato fondamentali</b>                    | <b>3</b>  |
| 2.1      | cpu_t e memaddr . . . . .                                               | 3         |
| 2.2      | pteEntry_t — entry di Page Table . . . . .                              | 4         |
| 2.3      | context_t — contesto per il pass-up . . . . .                           | 4         |
| 2.4      | support_t — la “Support Structure” . . . . .                            | 4         |
| 2.5      | swap_t — descrittore di frame nello Swap Pool . . . . .                 | 4         |
| 2.6      | pcb_t — il Process Control Block . . . . .                              | 4         |
| 2.7      | semd_t — descrittore di semaforo attivo . . . . .                       | 5         |
| <b>3</b> | <b>Tipi forniti dalla SDK <math>\mu</math>RISCV (uriscv/types.h)</b>    | <b>5</b>  |
| 3.1      | state_t — lo stato del processore . . . . .                             | 5         |
| 3.2      | passupvector_t — il Pass Up Vector . . . . .                            | 6         |
| 3.3      | dtpreg_t e termreg_t — registri dei device . . . . .                    | 6         |
| <b>4</b> | <b>headers/listx.h — liste concatenate generiche</b>                    | <b>7</b>  |
| 4.1      | L’idea: liste “intrusive” . . . . .                                     | 7         |
| 4.2      | offsetof e container_of . . . . .                                       | 7         |
| 4.3      | Inizializzazione . . . . .                                              | 8         |
| 4.4      | Inserimento . . . . .                                                   | 8         |
| 4.5      | Rimozione . . . . .                                                     | 8         |
| 4.6      | Test e navigazione . . . . .                                            | 8         |
| 4.7      | Macro di iterazione . . . . .                                           | 9         |
| <b>5</b> | <b>phase1/pcb.c — Process Control Block</b>                             | <b>9</b>  |
| 5.1      | Il file header phase1/headers/pcb.h . . . . .                           | 9         |
| 5.2      | Variabili statiche del modulo . . . . .                                 | 10        |
| 5.3      | initPcbs — inizializzazione . . . . .                                   | 10        |
| 5.4      | allocPcb — allocazione . . . . .                                        | 11        |
| 5.5      | freePcb — deallocazione . . . . .                                       | 11        |
| 5.6      | Gestione delle code di processi (ready queue e code semaforo) . . . . . | 11        |
| 5.7      | Gestione dell’albero dei processi . . . . .                             | 13        |
| <b>6</b> | <b>phase1/asl.c — Active Semaphore List</b>                             | <b>14</b> |
| 6.1      | Il file header phase1/headers/asl.h . . . . .                           | 14        |
| 6.2      | Cos’è la ASL e perché serve . . . . .                                   | 14        |
| 6.3      | Variabili globali del modulo . . . . .                                  | 14        |
| 6.4      | initASL . . . . .                                                       | 15        |

|      |                                                                                            |    |
|------|--------------------------------------------------------------------------------------------|----|
| 6.5  | <code>insertBlocked</code> — blocco di un processo su un semaforo . . . . .                | 15 |
| 6.6  | <code>removeBlocked</code> — sblocco del primo processo in coda . . . . .                  | 16 |
| 6.7  | <code>outBlocked</code> — rimozione mirata (non necessariamente in testa) . . . . .        | 16 |
| 6.8  | <code>headBlocked</code> — ispezione senza rimozione . . . . .                             | 17 |
| 7    | <code>klog.c</code> — buffer di log circolare per il debug . . . . .                       | 17 |
| 8    | Relazioni tra le strutture: la “mappa” d’insieme . . . . .                                 | 18 |
| 9    | Il test driver ufficiale: <code>phase1/p1test.c</code> . . . . .                           | 19 |
| 9.1  | Meccanismo di segnalazione . . . . .                                                       | 19 |
| 9.2  | Cosa verifica, in ordine . . . . .                                                         | 19 |
| 10   | Sistema di build: <code>CMakeLists.txt</code> e <code>config_machine.json</code> . . . . . | 20 |
| 10.1 | <code>CMakeLists.txt</code> — il target <code>phase1</code> . . . . .                      | 20 |
| 10.2 | <code>config_machine.json</code> — la configurazione dell’emulatore . . . . .              | 20 |

---

## 1 Introduzione generale alla Fase 1

La **Phase 1** di PandOSsh non è ancora un sistema operativo: non esistono scheduler, non esistono eccezioni gestite, non gira nulla in autonomia. È il livello più basso costruito *sopra* il firmware dell’emulatore  $\mu$ RISCv (che fa da BIOS), e fornisce esclusivamente **strutture dati di servizio** che i livelli superiori (Nucleus in Phase 2, Support Level in Phase 3) utilizzeranno per implementare i concetti di processo, sincronizzazione e scheduling.

Il codice della Fase 1 è interamente contenuto in due moduli:

- `phase1/pcb.c` — gestione dei **Process Control Block** (PCB): allocazione/deallocazione, code di processi, albero dei processi.
- `phase1/asl.c` — gestione della **Active Semaphore List** (ASL): la struttura che tiene traccia di quali processi sono bloccati su quale semaforo.

Entrambi i moduli si appoggiano a una libreria di liste concatenate generica (`headers/listx.h`, un sottoinsieme delle liste del kernel Linux) e alle definizioni di tipo in `headers/types.h`.

**Principio cardine: niente allocazione dinamica.** In tutta la Fase 1 (e in tutto il progetto) non compare mai `malloc`. Il numero massimo di processi (`MAXPROC`, 20) e di semafori attivi è fissato a tempo di compilazione: tutte le strutture sono array statici, gestiti tramite *free list*. Questo è un requisito tipico dei sistemi operativi “didattici in stile Pandos”: un kernel deve avere un comportamento **deterministico**, senza frammentazione della memoria e senza il rischio che un’allocazione fallisca in un momento critico.

**Perché liste doppiamente concatenate circolari?** Sia i PCB sia i semafori attivi sono organizzati con la stessa tecnica: una `struct list_head` embedded nella struttura dati, gestita con le funzioni generiche di `listx.h`. I vantaggi:

- inserimento e rimozione in  $O(1)$ , qualunque sia la posizione (testa, coda, elemento intermedio);
- nessun caso speciale per lista vuota o con un solo elemento (la sentinella `head` è essa stessa un nodo, quindi il codice che manipola `next/prev` è identico in ogni caso);
- una stessa struttura (es. `pcb_t`) può appartenere contemporaneamente a più liste diverse (free list, ready queue, coda di un semaforo, lista dei figli...) semplicemente avendo più campi `list_head`.

## 2 headers/types.h — i tipi di dato fondamentali

Questo header definisce *tutte* le strutture dati condivise da tutto il progetto (non solo dalla Fase 1). È importante conoscerle a menadito perché compaiono ovunque nel resto del kernel.

```
1 typedef signed int cpu_t;
2 typedef unsigned int memaddr;
3
4 /* Page Table Entry descriptor */
5 typedef struct pteEntry_t
6 {
7     unsigned int pte_entryHI;
8     unsigned int pte_entryLO;
9 } pteEntry_t;
10
11 /* Support level context */
12 typedef struct context_t
13 {
14     unsigned int stackPtr;
15     unsigned int status;
16     unsigned int pc;
17 } context_t;
18
19 /* Support level descriptor */
20 typedef struct support_t
21 {
22     int sup_asid;
23     state_t sup_exceptState[2];
24     context_t sup_exceptContext[2];
25     pteEntry_t sup_privatePgTbl[USERPGTBLSIZE];
26     unsigned int sup_stackTLB[500];
27     unsigned int sup_stackGen[500];
28     struct list_head s_list;
29 } support_t;
30
31 /* Page swap pool information structure type */
32 typedef struct swap_t
33 {
34     int sw_asid;
35     int sw_pageNo;
36     pteEntry_t *sw_pte;
37 } swap_t;
38
39 /* process table entry type */
40 typedef struct pcb_t {
41     struct list_head p_list;
42
43     struct pcb_t *p_parent;
44     struct list_head p_child;
45     struct list_head p_sib;
46
47     state_t p_s;
48     cpu_t p_time;
49
50     int *p_semAdd;
51     support_t *p_supportStruct;
52
53     int p_prio;
54     int p_pid;
55 } pcb_t, *pcb_PTR;
56
57 /* semaphore descriptor (SEMD) data structure */
58 typedef struct semd_t {
59     int *s_key;
60     struct list_head s_procq;
61     struct list_head s_link;
62 } semd_t, *semd_PTR;
```

### 2.1 cpu\_t e memaddr

Due semplici typedef per rendere il codice autodocumentante: `cpu_t` (int con segno) misura quantità di tempo CPU (letto dal TOD, il Time-Of-Day clock), `memaddr` (unsigned int) rappresenta un

indirizzo di memoria a 32 bit.

## 2.2 pteEntry\_t — entry di Page Table

Rappresenta *una* riga della page table usata dalla memoria virtuale (Fase 3): `pte_entryHI` contiene VPN (Virtual Page Number) e ASID, `pte_entryLO` contiene PFN (Physical Frame Number) più i bit di controllo (`V` = valida, `D` = dirty/scrivibile). Non è usata dalla Fase 1, ma è dichiarata qui perché fa parte del “vocabolario” di tipi di tutto il progetto.

## 2.3 context\_t — contesto per il pass-up

Tripletta (*stack pointer, status, program counter*) che descrive *dove* il Nucleus deve saltare quando passa un’eccezione a un livello superiore (Support Level). Viene caricata dall’istruzione `LDCXT` in `passUpOrDie` (Fase 2).

## 2.4 support\_t — la “Support Structure”

È la struttura che collega Nucleus e Support Level (dettagliata in Fase 3): contiene l’ASID del processo, due slot di stato salvato (`sup_exceptState[2]`, uno per i page fault e uno per le altre eccezioni), due context di ripartenza (`sup_exceptContext[2]`), la Page Table privata del processo utente (32 entry) e due stack dedicati agli handler (`sup_stackTLB`, `sup_stackGen`, 500 word ciascuno). Ogni `pcb_t` ha un puntatore opzionale a una `support_t` (`p_supportStruct`): se è `NULL` il processo è un processo “puro kernel” (Fase 2), altrimenti è una U-proc (Fase 3).

## 2.5 swap\_t — descrittore di frame nello Swap Pool

Usato solo in Fase 3: per ogni frame fisico dello Swap Pool ricorda quale ASID e quale pagina logica vi risiedono, e un puntatore alla PTE corrispondente (per poterla invalidare rapidamente in caso di sfratto).

## 2.6 pcb\_t — il Process Control Block

Il cuore della Fase 1. Campo per campo:

| Campo           | Significato                                                                                                                                                                                                                                                                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| p_list          | nodo di lista usato per inserire il PCB in <i>una</i> coda alla volta: la free list dei PCB liberi, la ready queue, oppure la coda dei processi bloccati su un semaforo ( <b>s_procq</b> in <b>semd_t</b> ). Un PCB non può stare in due di queste code contemporaneamente, quindi riutilizzare lo stesso campo per tutte e tre è sicuro e risparmia memoria. |
| p_parent        | puntatore al PCB del processo padre (NULL per il processo radice).                                                                                                                                                                                                                                                                                            |
| p_child         | testa della lista dei figli di <i>questo</i> processo.                                                                                                                                                                                                                                                                                                        |
| p_sib           | nodo con cui questo PCB si aggancia alla lista <b>p_child</b> del proprio padre (lista dei “fratelli”).                                                                                                                                                                                                                                                       |
| p_s             | <b>state_t</b> : l'intero stato del processore (registri, PC, status, ecc.) salvato/ripristinato a ogni context switch.                                                                                                                                                                                                                                       |
| p_time          | tempo di CPU accumulato dal processo (usato dalla syscall GETTIME).                                                                                                                                                                                                                                                                                           |
| p_semAdd        | puntatore intero: se non NULL, indirizzo del semaforo su cui il processo è bloccato (altrimenti il processo è pronto o in esecuzione).                                                                                                                                                                                                                        |
| p_supportStruct | puntatore alla <b>support_t</b> del processo, o NULL se non ne ha una.                                                                                                                                                                                                                                                                                        |
| p_prio          | priorità ( <b>PROCESS_PRIO_LOW</b> =0 o <b>PROCESS_PRIO_HIGH</b> =1) usata dallo scheduler.                                                                                                                                                                                                                                                                   |
| p_pid           | identificatore del processo, assegnato in ordine crescente da <b>allocPcb</b> .                                                                                                                                                                                                                                                                               |

**Da notare:** **p\_list** è usato per *tre* scopi diversi in tempi diversi (free list / ready queue / coda semaforo) ma mai simultaneamente: è un classico esempio di riuso di un campo di collegamento generico offerto dalla libreria di liste intrusive.

## 2.7 semd\_t — descrittore di semaforo attivo

- **s\_key**: puntatore che identifica *univocamente* il semaforo (in pratica l'indirizzo della variabile intera che rappresenta il semaforo, es. un elemento dell'array **devSems**).
- **s\_procq**: testa della coda dei PCB bloccati su questo semaforo (i nodi agganciati sono i campi **p\_list** dei PCB).
- **s\_link**: nodo con cui il descrittore si aggancia alla ASL (**semd\_h**) oppure alla free list dei descrittori (**semdFree\_h**).

## 3 Tipi forniti dalla SDK $\mu$ RISCv (**uriscv/types.h**)

**headers/types.h** inizia con **#include <uriscv/types.h>**: un header di sistema (in **/usr/include/uriscv/**, **non** scritto dal progetto) che definisce i tipi legati direttamente all'hardware emulato. Questi tipi compaiono in *ogni* listato dei tre documenti (**p\_s**, **savedState**, **exState**...) ma non sono mai dichiarati nei sorgenti del progetto: vale la pena vederli una volta per tutte qui, perché altrimenti campi come **->reg\_a0** o **->cause** restano “magici”.

### 3.1 state\_t — lo stato del processore

```

1 #define STATE_GPR_LEN 32
2
3 typedef struct state {
4     unsigned int entry_hi;

```

```

5     unsigned int cause;
6     unsigned int status;
7     unsigned int pc_epc;
8     unsigned int mie;
9     unsigned int gpr[STATE_GPR_LEN];
10 } state_t;
11
12 /* alias per i 32 registri general-purpose (convenzione RISC-V) */
13 #define reg_sp gpr[2]
14 #define reg_a0 gpr[24]
15 #define reg_a1 gpr[25]
16 #define reg_a2 gpr[26]
17 #define reg_a3 gpr[27]
18 /* ... reg_ra, reg_gp, reg_t0..t6, reg_s0..s11, reg_a4..a7 ... */

```

È la struttura **più importante di tutto il progetto**: rappresenta l'intero stato del processore in un dato istante, ed è ciò che viene salvato/ripristinato a ogni context switch, eccezione o interrupt.

- **entry\_hi**: registro TLB con VPN (indirizzo virtuale) e ASID — rilevante nei page fault (Fase 3).
- **cause**: il registro che spiega *perché* è avvenuta l'eccezione (bit 31 = interrupt; altrimenti ExcCode nei bit bassi). È il primo campo letto da `exceptionHandler`.
- **status**: i bit di modalità e abilitazione interrupt (MPP = kernel/user mode, MIE/MPIE = interrupt correnti/precedenti abilitati).
- **pc\_epc**: il program counter, cioè l'indirizzo dell'istruzione a cui riprendere l'esecuzione.
- **mie**: maschera di quali sorgenti di interrupt sono abilitate.
- **gpr[32]**: i 32 registri general-purpose della RISC-V (x0–x31). Il progetto non li indicizza mai numericamente, ma tramite le macro `reg_sp`, `reg_a0–reg_a3` (usate per leggere il numero e gli argomenti di una SYSCALL, esattamente come a0–a3 nella convenzione di chiamata RISC-V), ecc.

Nel PCB, il campo `p_s` (Sezione 2.6) è di tipo `state_t`: è la copia persistente dello stato di un processo mentre non sta girando. Quando un'eccezione avviene, il **firmware** (non il Nucleus) salva automaticamente lo stato corrente in un'area fissa di memoria (`BIOSDATAPAGE`, si veda il documento di Fase 2): è da lì che `exceptionHandler` legge `savedState`.

### 3.2 passupvector\_t — il Pass Up Vector

```

1 typedef struct passupvector {
2     unsigned int tlb_refill_handler;
3     unsigned int tlb_refill_stackPtr;
4     unsigned int exception_handler;
5     unsigned int exception_stackPtr;
6 } passupvector_t;

```

Quattro interi che il Nucleus scrive **una sola volta**, in `initial.c::main` (si veda il documento di Fase 2), a un indirizzo fisso (`PASSUPVECTOR`, `0x0FFF900`) letto dal firmware: dicono all'hardware dove saltare (`_handler`) e quale stack usare (`_stackPtr`) rispettivamente per un evento di TLB-Refill e per qualunque altra eccezione. È il meccanismo con cui il Nucleus si “registra” presso il firmware come gestore di tutte le eccezioni.

### 3.3 dtpreg\_t e termreg\_t — registri dei device

```

1 /* Device "semplici": disk, flash, printer */
2 typedef struct dtpreg {
3     unsigned int status;
4     unsigned int command;
5     unsigned int data0;
6     unsigned int data1;
7 } dtpreg_t;
8

```

```

9  /* Terminali: due sotto-canali indipendenti (ricezione e trasmissione) */
10 typedef struct termreg {
11     unsigned int  recv_status;
12     unsigned int  recv_command;
13     unsigned int  transm_status;
14     unsigned int  transm_command;
15 } termreg_t;

```

Due modi di interpretare il blocco di registri hardware di un device (l'indirizzo di partenza si calcola con la macro `DEV_REG_ADDR(line, dev)`, anch'essa fornita dalla SDK in `uriscv/arch.h`). I device “semplici” (disk, flash, printer) hanno un'unica coppia status/comando (più due registri dati, `data0/data1`, usati per il DMA: si veda `flashOperation` in Fase 3, che scrive l'indirizzo del frame in `data0`). I terminali, avendo due canali fisicamente indipendenti, raddoppiano la coppia: `recv_*` per la ricezione, `transm_*` per la trasmissione — esattamente la distinzione già introdotta a proposito di `TERM_TX_SEM/TERM_RX_SEM` (Fase 2) e usata da `writeTerminal/readTerminal` (Fase 3). Si noti che `interrupts.c` (Fase 2) non usa questi due `typedef` per nome, ma ottiene lo stesso effetto indicizzando manualmente un array di `unsigned int` agli stessi offset (si vedano le macro `DEV_STATUS/DEV_COMMAND/TERM_RECV_STATUS...` nel documento di Fase 2): risultato identico, stile diverso.

## 4 headers/listx.h — liste concatenate generiche

Questo file è un sottoinsieme (semplificato) delle liste doppiamente concatenate circolari del kernel Linux (`include/linux/list.h`). È la base tecnica su cui si costruiscono *tutte* le code del progetto: ready queue, code dei semafori, lista dei figli, free list dei PCB e dei `semd_t`.

### 4.1 L'idea: liste “intrusive”

Invece di una lista generica che contiene puntatori a dati arbitrari (come una `List<T>` ad alto livello), qui la lista è *intrusiva*: un campo `struct list_head` viene inserito *dentro* la struttura dati che si vuole collegare (es. `p_list` dentro `pcb_t`). La lista collega direttamente questi campi tra loro; per risalire alla struttura dati che li contiene si usa la macro `container_of`.

```

1  struct list_head {
2      struct list_head *next, *prev;
3  };

```

Una lista è quindi semplicemente un nodo con due puntatori. Una lista *vuota* è un nodo il cui `next` e `prev` puntano *a se stesso*: questo è il trucco che elimina tutti i casi speciali.

### 4.2 offsetof e container\_of

```

1  #define offsetof(TYPE, MEMBER) ((size_t)(&((TYPE *)0)->MEMBER))
2
3  #define container_of(ptr, type, member)                                \
4      ({                                                                    \
5          const typeof(((type *)0)->member) *__mptr = (ptr);              \
6          (type *)((char *)__mptr - offsetof(type, member));              \
7      })

```

- `offsetof(TYPE, MEMBER)` calcola, senza mai istanziare davvero una `TYPE`, la distanza in byte dall'inizio della struttura al campo `MEMBER`: finge che una `TYPE` sia allocata all'indirizzo 0 e legge l'indirizzo (che numericamente coincide con l'offset) del campo richiesto.
- `container_of(ptr, type, member)` fa l'operazione inversa di “prendere l'indirizzo di un campo e tornare al contenitore”: dato un puntatore `ptr` a un campo `member` di tipo `type`, sottrae l'offset del campo dall'indirizzo per ottenere il puntatore alla struttura intera. È così che, ad esempio, da un `struct list_head *` estratto dalla ready queue si risale al `pcb_t*` che lo contiene: `container_of(pos, pcb_t, p_list)`.

### 4.3 Inizializzazione

```
1 #define LIST_HEAD_INIT(name) { &(amp;name), &(name) }
2 #define LIST_HEAD(name) struct list_head name = LIST_HEAD_INIT(name)
3
4 static inline void INIT_LIST_HEAD(struct list_head *list) {
5     list->next = list;
6     list->prev = list;
7 }
```

LIST\_HEAD\_INIT/LIST\_HEAD sono macro usate per dichiarare e inizializzare, in un colpo solo, una variabile lista vuota a livello statico. INIT\_LIST\_HEAD è la funzione (inline) equivalente da chiamare a runtime su una lista già dichiarata: imposta entrambi i puntatori sulla lista stessa, rendendola una lista vuota valida.

### 4.4 Inserimento

```
1 static inline void __list_add(struct list_head *new, struct list_head *prev, struct
2     list_head *next) {
3     next->prev = new;
4     new->next = next;
5     new->prev = prev;
6     prev->next = new;
7 }
8
9 static inline void list_add(struct list_head *new, struct list_head *head) {
10     __list_add(new, head, head->next);
11 }
12
13 static inline void list_add_tail(struct list_head *new, struct list_head *head) {
14     __list_add(new, head->prev, head);
15 }
```

\_\_list\_add è la primitiva di base: inserisce *new* tra due nodi già collegati (prev e next), aggiornando i 4 puntatori coinvolti. È  $O(1)$  e funziona identicamente sia che la lista sia vuota, con un elemento o con molti.

- list\_add(new, head): inserisce *new subito dopo head*, cioè in **testa** alla lista (prev=head, next=head->next).
- list\_add\_tail(new, head): inserisce *new subito prima di head*, cioè in **coda** (prev=head->prev, next=head). È la funzione più usata nel progetto (free list, code FIFO, inserimento dei figli).

### 4.5 Rimozione

```
1 static inline void __list_del(struct list_head *prev, struct list_head *next) {
2     next->prev = prev;
3     prev->next = next;
4 }
5
6 static inline void list_del(struct list_head *entry) {
7     __list_del(entry->prev, entry->next);
8     entry->next = entry;
9     entry->prev = entry;
10 }
```

\_\_list\_del ricollega direttamente prev e next “scavalcando” il nodo che sta in mezzo. list\_del(entry) la usa per staccare entry dalla lista in cui si trova, e poi **reimposta entry come lista vuota** (punta a se stesso): questo evita puntatori pendenti (dangling) se qualcuno riusa quel nodo per errore prima di reinserirlo altrove.

### 4.6 Test e navigazione

```

1 static inline int list_is_last(const struct list_head *list, const struct list_head *head) {
2     return list->next == head;
3 }
4
5 static inline int list_empty(const struct list_head *head) {
6     return head->next == head;
7 }
8
9 static inline struct list_head *list_next(const struct list_head *current) {
10     if (list_empty(current)) return NULL;
11     else return current->next;
12 }
13
14 static inline struct list_head *list_prev(const struct list_head *current) {
15     if (list_empty(current)) return NULL;
16     else return current->prev;
17 }

```

`list_empty` sfrutta esattamente l'invariante di inizializzazione: una lista vuota ha `head->next == head`. `list_is_last` controlla se `list` è l'ultimo nodo prima della sentinella `head`. `list_next`/`list_prev` sono getter di comodo (usati poco nel progetto, che preferisce le macro di iterazione sottostanti).

## 4.7 Macro di iterazione

```

1 #define list_for_each(pos, head) \
2     for (pos = (head)->next; pos != (head); pos = pos->next)
3
4 #define list_for_each_prev(pos, head) \
5     for (pos = (head)->prev; pos != (head); pos = pos->prev)
6
7 #define list_for_each_entry(pos, head, member) \
8     for (pos = container_of((head)->next, typeof(*pos), member); \
9         &pos->member != (head); \
10         pos = container_of(pos->member.next, typeof(*pos), member))
11
12 #define list_for_each_entry_reverse(pos, head, member) \
13     for (pos = container_of((head)->prev, typeof(*pos), member); \
14         &pos->member != (head); \
15         pos = container_of(pos->member.prev, typeof(*pos), member))

```

- `list_for_each(pos, head)`: itera sui *nodi* `list_head` grezzi, partendo dal primo elemento reale (`head->next`) e fermandosi quando si torna alla sentinella. Usata in `as1.c` e `pcb.c` quando serve confrontare `pos` con l'indirizzo di un campo specifico (es. `outProcQ`, `outBlocked`).
- `list_for_each_entry(pos, head, member)`: variante di comodo che restituisce direttamente il puntatore alla *struttura contenitore* (tramite `container_of` a ogni iterazione), evitando di dover fare la conversione manualmente nel corpo del ciclo. È quella usata in `as1.c` per scorrere `semd_h` (`semd_t *s`).
- Le due varianti `_reverse` percorrono la lista all'indietro partendo da `head->prev`; non sono usate nel codice del progetto ma sono disponibili nella libreria.

## 5 phase1/pcb.c — Process Control Block

### 5.1 Il file header phase1/headers/pcb.h

```

1 // SPDX-FileCopyrightText: 2022 Luca Bassi, Gaia Clerici, Mirco Dondi, Fabio Gaiba
2 //
3 // SPDX-License-Identifier: GPL-3.0-or-later
4
5 #ifndef PCB_H_INCLUDED
6 #define PCB_H_INCLUDED
7
8 #include "../headers/listx.h"
9 #include "../headers/types.h"
10

```

```

11 void initPcbs();
12 void freePcb(pcb_t* p);
13 pcb_t* allocPcb();
14
15 void mkEmptyProcQ(struct list_head* head);
16 int emptyProcQ(struct list_head* head);
17 void insertProcQ(struct list_head* head, pcb_t* p);
18 pcb_t* headProcQ(struct list_head* head);
19 pcb_t* removeProcQ(struct list_head* head);
20 pcb_t* outProcQ(struct list_head* head, pcb_t* p);
21
22 int emptyChild(pcb_t* p);
23 void insertChild(pcb_t* prnt, pcb_t* p);
24 pcb_t* removeChild(pcb_t* p);
25 pcb_t* outChild(pcb_t* p);
26
27 #endif

```

`pcb.h` è l'**interfaccia pubblica** del modulo: dichiara i prototipi delle 13 funzioni analizzate più sotto (nel file originale ogni prototipo è preceduto da un commento su una riga che ne riassume lo scopo — lo stesso testo usato come base per le spiegazioni di questa sezione), include `listx.h` (per `struct list_head`) e `types.h` (per `pcb_t`), e non definisce *nulla*: nessuna variabile, nessun corpo di funzione. È questa separazione header/implementazione che permette a `phase2` e `phase3` di usare i PCB (`#include "../phase1/headers/pcb.h"`) senza conoscere né poter accedere a `pcbFree_h`, `pcbFree_table` o `next_pid` (dichiarati `static` in `pcb.c`, quindi invisibili fuori da quel file): l'unico modo per manipolare i PCB dall'esterno è tramite queste 13 funzioni.

È interessante notare che la testata **SPDX** attribuisce questo file (e il gemello `as1.h`) a un altro gruppo di autori (2022): è l'**interfaccia data dal corso** (il “contratto” che l'implementazione deve rispettare, ereditato da un precedente progetto PandOS didattico open-source), mentre `pcb.c` e `as1.c` — le implementazioni vere e proprie analizzate in questo documento — sono il lavoro originale di questo team. Distinguere le due cose è utile se l'esaminatore chiede “questo file l'avete scritto voi?”.

## 5.2 Variabili statiche del modulo

```

1 static struct list_head pcbFree_h;
2 static pcb_t pcbFree_table[MAXPROC];
3 static int next_pid = 1;

```

- `pcbFree_table[MAXPROC]`: l'unico array di PCB dell'intero sistema (20 elementi, `MAXPROC=20` in `const.h`). *Ogni* processo che esisterà mai durante l'esecuzione userà uno di questi 20 slot: non c'è altro modo per ottenere un PCB.
- `pcbFree_h`: testa della free list che collega, tramite il campo `p_list`, tutti gli slot di `pcbFree_table` attualmente *non* in uso.
- `next_pid`: contatore globale monotono crescente, usato per assegnare un PID univoco a ogni PCB allocato. Parte da 1 (0 è riservato per rappresentare “nessun PID”/errore in alcune chiamate, es. `GETPROCESSID`). **Nota:** i PID non vengono mai riusati anche se un processo termina, quindi crescono indefinitamente entro i limiti di un `int` — comportamento accettabile perché il numero di processi creabili in una sessione è comunque limitato dalla dimensione della RAM simulata e dalla durata delle demo.

## 5.3 `initPcbs` — inizializzazione

```

1 void initPcbs() {
2     int i;
3     INIT_LIST_HEAD(&pcbFree_h);
4
5     for (i = 0; i < MAXPROC; i++) {
6         INIT_LIST_HEAD(&pcbFree_table[i].p_list);
7         pcbFree_table[i].p_pid = 0;

```

```

8     list_add_tail(&pcbFree_table[i].p_list, &pcbFree_h);
9 }
10 }

```

Chiamata *una sola volta* all'avvio del sistema (da `initial.c`). Inizializza la sentinella `pcbFree_h` come lista vuota, poi per ciascuno dei 20 slot dell'array statico: inizializza il proprio `p_list` come nodo isolato e lo accoda (`list_add_tail`) alla free list. Al termine, `pcbFree_h` contiene tutti e 20 gli slot, nello stesso ordine dell'array. Costo:  $O(\text{MAXPROC})$ , eseguito una volta sola al boot.

## 5.4 allocPcb — allocazione

```

1  pcb_t* allocPcb() {
2      if (list_empty(&pcbFree_h)) return NULL;
3
4      struct list_head *pos = pcbFree_h.next;
5      list_del(pos);
6
7      pcb_t *new_pcb = container_of(pos, pcb_t, p_list);
8      new_pcb->p_pid = next_pid++;
9      INIT_LIST_HEAD(&new_pcb->p_list);
10     INIT_LIST_HEAD(&new_pcb->p_child);
11     INIT_LIST_HEAD(&new_pcb->p_sib);
12     new_pcb->p_parent      = NULL;
13     new_pcb->p_semAdd      = NULL;
14     new_pcb->p_supportStruct = NULL;
15     new_pcb->p_time        = 0;
16     new_pcb->p_prio        = 0;
17
18     return new_pcb;
19 }

```

Se la free list è vuota (tutti i 20 PCB sono in uso: limite massimo di processi concorrenti raggiunto), ritorna `NULL` — il chiamante (in Fase 2, la syscall `CREATEPROCESS`) deve gestire questo fallimento restituendo un errore al processo padre, *senza mai* bloccarsi o andare in crash. Altrimenti:

1. preleva il *primo* nodo della free list (`pcbFree_h.next`) e lo stacca con `list_del`;
2. risale al `pcb_t` contenitore con `container_of`;
3. gli assegna un nuovo PID univoco (`next_pid++`, post-incremento: usa il valore corrente e *poi* incrementa);
4. reinizializza da zero *tutti* i campi rilevanti: le tre liste intrusive (`p_list`, `p_child`, `p_sib`) come liste vuote, e azzerava puntatore al padre, puntatore al semaforo, puntatore alla support struct, tempo CPU e priorità.

Questo azzeramento sistematico è cruciale: un PCB può essere stato usato in precedenza da un altro processo (poi terminato e liberato), quindi conterrebbe “spazzatura” residua se non venisse ripulito esplicitamente — non essendoci `malloc`, non c'è nemmeno uno zeroing implicito come con `calloc`.

## 5.5 freePcb — deallocazione

```

1  void freePcb(pcb_t* p) {
2      if (p != NULL) list_add_tail(&p->p_list, &pcbFree_h);
3  }

```

Semplicemente riaggancia `p` in coda alla free list tramite il suo campo `p_list` (che a questo punto deve essere già stato staccato da qualunque altra coda in cui si trovasse — responsabilità del chiamante, tipicamente `terminateProcess` in Fase 2).  $O(1)$ .

## 5.6 Gestione delle code di processi (ready queue e code semaforo)

```

1  void mkEmptyProcQ(struct list_head* head) {
2      INIT_LIST_HEAD(head);
3  }

```

```

4
5 int emptyProcQ(struct list_head* head) {
6     return list_empty(head);
7 }
8
9 void insertProcQ(struct list_head* head, pcb_t* p) {
10     struct list_head *pos;
11     pcb_t *iter;
12     list_for_each(pos, head) {
13         iter = container_of(pos, pcb_t, p_list);
14         if (p->p_prio > iter->p_prio) {
15             __list_add(&p->p_list, pos->prev, pos);
16             return;
17         }
18     }
19     list_add_tail(&p->p_list, head);
20 }
21
22 pcb_t* headProcQ(struct list_head* head) {
23     if (list_empty(head)) return NULL;
24     return container_of(head->next, pcb_t, p_list);
25 }
26
27 pcb_t* removeProcQ(struct list_head* head) {
28     if (list_empty(head)) return NULL;
29     struct list_head *first = head->next;
30     list_del(first);
31     INIT_LIST_HEAD(first);
32     return container_of(first, pcb_t, p_list);
33 }
34
35 pcb_t* outProcQ(struct list_head* head, pcb_t* p) {
36     struct list_head *pos;
37     pcb_t *iter;
38     list_for_each(pos, head) {
39         iter = container_of(pos, pcb_t, p_list);
40         if (iter == p) {
41             list_del(pos);
42             return p;
43         }
44     }
45     return NULL;
46 }

```

- `mkEmptyProcQ(head)`: banale wrapper su `INIT_LIST_HEAD`; crea/azzerava una coda di processi vuota. Usata da `initial.c` per la `readyQueue` globale.
- `emptyProcQ(head)`: wrapper su `list_empty`.
- `insertProcQ(head, p)`: questa è l'unica funzione della Fase 1 che introduce una politica di ordinamento. Scorre la coda dal primo elemento e, appena trova un elemento `iter` con priorità *strettamente minore* di `p`, inserisce `p` prima di `iter` (con l'inserimento a basso livello `__list_add`, per poterlo fare in mezzo alla lista). Se non trova nessun elemento con priorità minore (cioè tutti hanno priorità  $\geq$  quella di `p`, oppure la coda è vuota), lo accoda in fondo con `list_add_tail`. Risultato: la coda resta sempre ordinata per priorità *decrescente* in testa, e a parità di priorità l'ordine di inserimento è preservato (FIFO) — è esattamente ciò che serve per realizzare un **round-robin con priorità** nello scheduler di Fase 2: basta sempre estrarre la testa.
- `headProcQ(head)`: restituisce (senza rimuoverlo) il PCB in testa, cioè il primo `container_of` di `head->next`; `NULL` se la coda è vuota. Usata dallo scheduler per ispezionare il prossimo candidato senza consumarlo (es. nella gestione dello `YIELD`).
- `removeProcQ(head)`: rimuove e restituisce la testa. Stacca il primo nodo (`list_del`), lo reinizializza come lista a sé (`INIT_LIST_HEAD`, per igiene, così il PCB rimosso non conserva puntatori pendenti alla vecchia coda) e ne risale al `pcb_t`. Questa è la funzione che lo scheduler chiama per estrarre il *prossimo processo da eseguire*.

- `outProcQ(head, p)`: rimozione *mirata* di un PCB specifico da una coda arbitraria (non necessariamente in testa). Scorre linearmente la coda con `list_for_each` finché non trova il nodo il cui contenitore è `p`, poi lo stacca con `list_del` e lo ritorna; se `p` non è nella coda, ritorna `NULL`. È  $O(n)$  nel caso peggiore (a differenza delle altre operazioni, tutte  $O(1)$ ): serve ad esempio quando un processo viene terminato mentre è ancora nella ready queue (in Fase 2, `terminateProcess` chiama sempre `outProcQ(&readyQueue, p)` per sicurezza, anche se `p` potrebbe non essere in quella coda in quel momento — la funzione in tal caso semplicemente non trova nulla e ritorna `NULL` senza effetti).

## 5.7 Gestione dell'albero dei processi

```

1 int emptyChild(pcb_t* p) {
2     return list_empty(&p->p_child);
3 }
4
5 void insertChild(pcb_t *prnt, pcb_t *p) {
6     p->p_parent = prnt;
7     list_add_tail(&p->p_sib, &prnt->p_child);
8 }
9
10 pcb_t* removeChild(pcb_t* p) {
11     if (!p || list_empty(&p->p_child)) return NULL;
12     struct list_head *first = p->p_child.next;
13     pcb_t *child = container_of(first, pcb_t, p_sib);
14     list_del(&child->p_sib);
15     INIT_LIST_HEAD(&child->p_sib);
16     child->p_parent = NULL;
17     return child;
18 }
19
20 pcb_t* outChild(pcb_t* p) {
21     if (!p || !p->p_parent) return NULL;
22     list_del(&p->p_sib);
23     INIT_LIST_HEAD(&p->p_sib);
24     p->p_parent = NULL;
25     return p;
26 }

```

La rappresentazione dell'albero dei processi è “a puntatore al padre + lista dei figli”, il classico schema *first-child, next-sibling* adattato alle liste intrusive:

- ogni PCB ha un *unico* puntatore `p_parent` al padre;
- ogni PCB ha una *lista* `p_child`, che è la testa della lista dei propri figli;
- ogni PCB si aggancia alla lista `p_child` del padre tramite il proprio campo `p_sib` (*sibling*, fratello).
- `emptyChild(p)`: vero se `p` non ha figli.
- `insertChild(prnt, p)`: rende `p` figlio di `prnt`: imposta `p->p_parent = prnt` e accoda `p->p_sib` in fondo a `prnt->p_child`. Chiamata dalla syscall `CREATEPROCESS` appena dopo aver allocato il nuovo PCB.
- `removeChild(p)`: rimuove e ritorna il *primo* figlio di `p` (quello inserito per primo, in testa alla lista), staccandolo (`list_del(&child->p_sib)`) e azzerandone il puntatore al padre. Se `p` non ha figli ritorna `NULL`. È usata da `terminateProcess` (Fase 2) in un ciclo `while ((child = removeChild(p)) != NULL) terminateProcess(child)`; per terminare *ricorsivamente e distruttivamente* tutto un sotto-albero: a ogni iterazione stacca un figlio (quindi la lista si accorcia) e lo termina ricorsivamente, il che a sua volta staccherà e terminerà i suoi discendenti, finché l'albero sotto `p` non è vuoto.
- `outChild(p)`: operazione *duale* di `removeChild`: data direttamente la vittima `p` (non il padre), la stacca dalla lista dei figli del *proprio* padre e azzerà `p_parent`. Se `p` non ha un padre (`p_parent == NULL`, es. è il processo radice, oppure è già stato staccato) ritorna `NULL` senza

fare nulla. È usata in `terminateProcess` *dopo* aver già staccato tutti i figli di `p` da `p` stesso, per staccare `p` dal proprio padre: se non lo si facesse, un processo che termina se stesso (o che viene terminato mentre non è più figlio “attivo”) resterebbe agganciato come figlio “fantasma” nella lista del padre, e una successiva terminazione del padre lo ritroverebbe (già liberato!) tramite `removeChild`, causando un uso di memoria invalida.

## 6 phase1/asl.c — Active Semaphore List

### 6.1 Il file header phase1/headers/asl.h

```
1 // SPDX-FileCopyrightText: 2022 Luca Bassi, Gaia Clerici, Mirco Dondi, Fabio Gaiba
2 //
3 // SPDX-License-Identifier: GPL-3.0-or-later
4
5 #ifndef ASL_H_INCLUDED
6 #define ASL_H_INCLUDED
7
8 #include "../headers/listx.h"
9 #include "../headers/types.h"
10
11 extern semd_t semd_table[MAXPROC];
12 extern struct list_head semdFree_h;
13 extern struct list_head semd_h;
14
15 void initASL();
16 int insertBlocked(int* semAdd, pcb_t* p);
17 pcb_t* removeBlocked(int* semAdd);
18 pcb_t* outBlocked(pcb_t* p);
19 pcb_t* headBlocked(int* semAdd);
20
21 #endif
```

Stessa struttura di `pcb.h`, con una differenza degna di nota: qui `semd_table`, `semdFree_h` e `semd_h` sono dichiarate `extern` (non `static` come i loro analoghi in `pcb.c`): sono quindi *tecnicamente* accessibili anche da fuori `asl.c`, a differenza dei PCB. Nella pratica nessun altro modulo del progetto le usa direttamente (Fase 2 e Fase 3 passano sempre attraverso le 5 funzioni pubbliche), ma la loro visibilità `extern` lascia aperta la possibilità — prevista dalla specifica originale di Pandos, da cui questa interfaccia deriva — di ispezionarle a fini di debug avanzato. Anche questo header porta la stessa attribuzione SPDX del 2022 discussa per `pcb.h`: fa parte della stessa interfaccia “data”, che `asl.c` implementa.

### 6.2 Cos’è la ASL e perché serve

Un semaforo, in questo progetto, è semplicemente un `int` (es. una cella dell’array `devSems` in Fase 2, oppure una variabile globale come `swapPoolSem` in Fase 3): non esiste una struttura dati “semaforo” associata a ciascuna variabile intera. Quando però un processo si blocca *su* un semaforo, serve un posto dove tenere la coda dei processi in attesa *per quel preciso indirizzo*. La **Active Semaphore List** (ASL) è esattamente questo: una lista di descrittori `semd_t`, uno per ogni semaforo che ha *almeno un* processo bloccato in questo momento (da cui “active”). Un semaforo che nessuno sta aspettando non ha nessun descrittore nella ASL: se il suo valore diventa negativo per la prima volta, un descrittore viene allocato al volo dalla free list; quando l’ultimo processo in attesa su di esso viene rimosso, il descrittore torna nella free list.

### 6.3 Variabili globali del modulo

```
1 semd_t semd_table[MAXPROC];
2 struct list_head semdFree_h;
3 struct list_head semd_h;
```

- `semd_table[MAXPROC]`: array statico di descrittori di semaforo. La dimensione è `MAXPROC` (20) e non un valore dedicato ai semafori perché, nella teoria classica di Pandos, nel caso peggiore *ogni* processo può essere bloccato su un semaforo *diverso*: non serve più di `MAXPROC` descrittori contemporaneamente attivi.
- `semdFree_h`: free list dei descrittori non in uso.
- `semd_h`: la ASL vera e propria, cioè la lista dei descrittori *attualmente attivi* (con almeno un processo in coda).

A differenza di `pcb.c`, qui non c'è dichiarazione `static`: sono dichiarate anche in `asl.h` con `extern`, quindi visibili da fuori il modulo (anche se in pratica solo `asl.c` le manipola).

## 6.4 initASL

```

1 void initASL() {
2     INIT_LIST_HEAD(&semdFree_h);
3     INIT_LIST_HEAD(&semd_h);
4     for (int i = 0; i < MAXPROC; i++) {
5         semd_table[i].s_key = NULL;
6         INIT_LIST_HEAD(&semd_table[i].s_procq);
7         list_add_tail(&semd_table[i].s_link, &semdFree_h);
8     }
9 }

```

Analoga a `initPcb`s: inizializza le due liste sentinella come vuote, poi per ciascuno dei 20 descrittori azzerla la chiave, inizializza la coda dei processi bloccati come vuota, e lo accoda alla free list tramite `s_link`. Chiamata una sola volta al boot, subito dopo `initPcb`s().

## 6.5 insertBlocked — blocco di un processo su un semaforo

```

1 int insertBlocked(int* semAdd, pcb_t* p) {
2     if (!semAdd || !p) return 1; // errore
3     semd_t* s;
4     list_for_each_entry(s, &semd_h, s_link) {
5         if (s->s_key == semAdd) {
6             list_add_tail(&p->p_list, &s->s_procq);
7             p->p_semAdd = semAdd;
8             return 0;
9         }
10    }
11    // semd non trovato, usa uno dalla free list
12    if (list_empty(&semdFree_h)) return 1; // nessun semd disponibile
13    s = container_of(semdFree_h.next, semd_t, s_link);
14    list_del(&s->s_link);
15    s->s_key = semAdd;
16    INIT_LIST_HEAD(&s->s_procq);
17    list_add_tail(&p->p_list, &s->s_procq);
18    p->p_semAdd = semAdd;
19    list_add_tail(&s->s_link, &semd_h);
20    return 0;
21 }

```

Logica in due fasi:

1. **Ricerca**: scorre `semd_h` con `list_for_each_entry` confrontando `s->s_key` con `semAdd`. Se trova un descrittore *già attivo* per quel semaforo, ci accoda semplicemente `p` (tramite `p->p_list`), imposta `p->p_semAdd = semAdd` e ritorna 0 (successo).
2. **Allocazione**: se nessun descrittore esiste ancora per quel semaforo (è la *prima* volta che qualcuno si blocca su di esso), ne preleva uno dalla free list (se disponibile, altrimenti errore 1: situazione limite che segnalerebbe un bug, dato che `MAXPROC` descrittori bastano sempre a coprire `MAXPROC` processi), lo configura con la chiave giusta e una coda vuota, ci accoda `p`, e lo sposta dalla free list alla ASL attiva (`list_add_tail(&s->s_link, &semd_h)`).

Nota: la ricerca è *lineare*,  $O(|\text{semd\_h}|)$ : nel caso peggiore  $O(\text{MAXPROC})$ . È accettabile perché il numero di semafori *attivi* contemporaneamente è comunque limitato dal numero di processi vivi, e la specifica del progetto non richiede una struttura ordinata più sofisticata (a differenza di alcune varianti storiche di Pandos che tengono la ASL ordinata per indirizzo per poter interrompere la ricerca in anticipo).

## 6.6 removeBlocked — sblocco del primo processo in coda

```

1 pcb_t* removeBlocked(int* semAdd) {
2     if (!semAdd) return NULL;
3     semd_t* s;
4     list_for_each_entry(s, &semd_h, s_link) {
5         if (s->s_key == semAdd) {
6             if (list_empty(&s->s_procq)) return NULL;
7             pcb_t* p = container_of(s->s_procq.next, pcb_t, p_list);
8             list_del(&p->p_list);
9             p->p_semAdd = NULL;
10            if (list_empty(&s->s_procq)) {
11                list_del(&s->s_link);
12                s->s_key = NULL;
13                list_add_tail(&s->s_link, &semdFree_h);
14            }
15            return p;
16        }
17    }
18    return NULL;
19 }

```

Trova il descrittore del semaforo `semAdd` (stessa ricerca lineare di sopra). Se la sua coda è vuota (situazione anomala: un descrittore attivo dovrebbe sempre avere almeno un processo, altrimenti sarebbe già tornato alla free list) ritorna NULL. Altrimenti:

1. preleva il *primo* PCB della coda (FIFO: chi si è bloccato per primo viene sbloccato per primo), lo stacca e azzerava il suo `p_semAdd` (segnalando che ora non è più bloccato su nulla);
2. **se dopo la rimozione la coda del semaforo è diventata vuota**, il descrittore non serve più: viene staccato dalla ASL e restituito alla free list. Questo è il meccanismo che mantiene la ASL “pulita”, contenente solo descrittori realmente attivi.

Chiamata dalla syscall `VERHOGEN` (V) in Fase 2, quando il valore del semaforo, dopo l’incremento, resta  $\leq 0$  (cioè c’era almeno un processo in attesa).

## 6.7 outBlocked — rimozione mirata (non necessariamente in testa)

```

1 pcb_t* outBlocked(pcb_t* p) {
2     if (!p || !p->p_semAdd) return NULL;
3     semd_t* s;
4     list_for_each_entry(s, &semd_h, s_link) {
5         if (s->s_key == p->p_semAdd) {
6             struct list_head* pos;
7             list_for_each(pos, &s->s_procq) {
8                 if (pos == &p->p_list) {
9                     list_del(&p->p_list);
10                    p->p_semAdd = NULL;
11                    if (list_empty(&s->s_procq)) {
12                        list_del(&s->s_link);
13                        s->s_key = NULL;
14                        list_add_tail(&s->s_link, &semdFree_h);
15                    }
16                    return p;
17                }
18            }
19            return NULL;
20        }
21    }
22    return NULL;
23 }

```

Rimuove **p** dalla coda del semaforo su cui è bloccato (letto da **p->p\_semAdd**, non passato esplicitamente: è **p** stesso a “sapere” su cosa è bloccato), *indipendentemente dalla sua posizione* nella coda (non necessariamente il primo). Doppio ciclo annidato: prima trova il descrittore giusto (uguale a **insertBlocked/removeBlocked**), poi scorre la sua coda di processi confrontando i nodi **p\_list** finché non trova esattamente **&p->p\_list**. Anche qui, se la coda si svuota dopo la rimozione, il descrittore torna alla free list.

È l'unica delle tre operazioni di rimozione a costo  $O(n \cdot m)$  nel caso peggiore (dove  $n$  = semafori attivi,  $m$  = lunghezza della coda), ma serve per un caso che le altre due coprono: quando un processo bloccato *da qualche parte* deve essere estratto senza sapere a priori la sua posizione. È usata in Fase 2 da **terminateProcess**: se il processo che si sta terminando risulta bloccato su un semaforo (**p->p\_semAdd != NULL**), bisogna toglierlo dalla coda di quel semaforo prima di liberarne il PCB, altrimenti resterebbe un puntatore pendente nella ASL verso un PCB ormai restituito alla free list.

## 6.8 headBlocked — ispezione senza rimozione

```

1 pcb_t* headBlocked(int* semAdd) {
2     if (!semAdd) return NULL;
3     semd_t* s;
4     list_for_each_entry(s, &semd_h, s_link) {
5         if (s->s_key == semAdd) {
6             if (list_empty(&s->s_procq)) return NULL;
7             return container_of(s->s_procq.next, pcb_t, p_list);
8         }
9     }
10    return NULL;
11 }

```

Come **removeBlocked** ma *senza* modificare nulla: trova il descrittore del semaforo e restituisce (senza toglierlo) il primo PCB della sua coda, oppure **NULL** se il semaforo non è attivo o la sua coda è vuota. Nel codice attuale del progetto non è invocata direttamente dai moduli di Fase 2/3 forniti (serve come primitiva di ispezione prevista dalla specifica standard di Pandos), ma fa parte dell'interfaccia pubblica di **asl.h** richiesta.

## 7 klog.c — buffer di log circolare per il debug

**klog.c** è compilato in **tutti e tre** i target del progetto (**CMakeLists.txt**: compare come primo file sorgente di **MultiPandOS\_phase1**, **MultiPandOS\_phase2** e **MultiPandOS\_phase3**), quindi va conosciuto anche se non appartiene concettualmente a una fase specifica.

```

1 #define KLOG_LINES      64
2 #define KLOG_LINE_SIZE  42
3
4 unsigned int klog_line_index = 0;
5 unsigned int klog_char_index = 0;
6 char        klog_buffer[KLOG_LINES][KLOG_LINE_SIZE] = {0};
7
8 void klog_print(char *str) {
9     while (*str != '\0') {
10         if (*str == '\n') { next_line(); str++; }
11         else { klog_buffer[klog_line_index][klog_char_index] = *str++; next_char(); }
12     }
13 }
14
15 void klog_print_dec(unsigned int num) { /* stampa un numero decimale 0..99 */ }
16 void klog_print_hex(unsigned int num) { /* stampa un numero in esadecimale */ }
17
18 static void next_char(void) {
19     if (++klog_char_index >= KLOG_LINE_SIZE) { klog_char_index = 0; next_line(); }
20 }
21 static void next_line(void) {
22     klog_line_index = (klog_line_index + 1) % KLOG_LINES;
23     klog_char_index = 0;
24     for (unsigned int i = 0; i < KLOG_LINE_SIZE; i++)

```

```

25 klog_buffer[klog_line_index][i] = ' ';
26 }

```

È una piccola libreria di logging **indipendente dal terminale virtuale**: invece di scrivere caratteri su un device (che richiede DOI0, quindi un Nucleus già funzionante e un processo che può bloccarsi), `klog_print` scrive direttamente in un array globale circolare `klog_buffer` (64 righe da 42 caratteri). Il buffer è pensato per essere **ispezionato dal debugger dell'emulatore** (tracciando l'indirizzo del simbolo `klog_buffer` nella finestra di memoria di uMPS3/uRISCV): permette di lasciare “tracce” di cosa sta succedendo nel kernel anche in situazioni in cui il terminale non è ancora disponibile o si sospetta che il problema sia proprio nel sottosistema di I/O che si starebbe usando per stampare a video.

- `klog_print(str)`: copia `str` carattere per carattere nella riga corrente, andando a capo (`next_line`) su ogni `'\n'` incontrato nella stringa sorgente.
- `klog_print_dec/klog_print_hex`: varianti per stampare un numero, rispettivamente in decimale (limitato per costruzione all'intervallo 0–99: il ramo `if(num >= 10)` gestisce due o più cifre in un ciclo, il ramo `else` gestisce esplicitamente il caso a una sola cifra) o in esadecimale (nessun limite, cifre 0–9A–F).
- `next_char/next_line` (`static`, private del modulo): gestiscono l'avanzamento circolare dentro la riga corrente e tra le righe (`% KLOG_LINES`), ripulendo con spazi la riga nuova per leggibilità quando si va a capo.

**Attenzione:** nel codice attuale non è mai invocato. Una ricerca di `klog_print` nell'intero albero dei sorgenti (`pcb.c`, `as1.c`, `initial.c`, `scheduler.c`, `exceptions.c`, `interrupts.c`, `initProc.c`, `vmSupport.c`, `sysSupport.c`) non produce alcun risultato: è compilato e linkato in ogni build, ma **nessuna chiamata a queste funzioni è presente** nel resto del kernel fornito. Il debug “a schermo” effettivamente utilizzato nel progetto passa invece dalle macro condizionali di `phase2/debug.h` (si veda il documento di Fase 2, che scrivono davvero sul terminale via i registri hardware). È comunque utile sapere spiegare *cosa fa e a cosa servirebbe* `klog.c` se l'esaminatore lo notasse tra i sorgenti compilati.

## 8 Relazioni tra le strutture: la “mappa” d'insieme

È utile fissare mentalmente come tutte queste liste si intreccino durante la vita di un processo:

1. Al boot, tutti i 20 PCB sono nella **free list** dei PCB (`pcbFree_h`) e tutti i 20 `semd_t` sono nella **free list** dei semafori (`semdFree_h`).
2. `allocPcb()` sposta un PCB dalla free list nel “mondo”: da quel momento il suo campo `p_list` è libero per essere agganciato altrove (Fase 2 lo inserisce subito nella ready queue).
3. Mentre un processo è **pronto**, il suo `p_list` è nella `readyQueue` (ordinata per priorità da `insertProcQ`).
4. Quando un processo esegue una P che lo blocca, `p_list` viene spostato (da `insertBlocked`) nella coda `s_procq` del descrittore `semd_t` giusto, allocandolo dalla free list dei semafori se necessario.
5. Una V (`removeBlocked`) o una terminazione forzata (`outBlocked`) rimuove il processo da quella coda e, se la coda diventa vuota, il `semd_t` torna alla free list dei semafori.
6. Parallelamente, ogni processo (tranne il primo) è agganciato tramite `p_sib` alla lista `p_child` del proprio padre: questo albero *coesiste* con le code sopra, è del tutto indipendente da esse (usa campi diversi) e serve solo a sapere “chi è figlio di chi”, per poter terminare interi sotto-alberi.
7. Alla terminazione, il PCB viene staccato da tutte le liste in cui si trova (`outChild` dal padre, `outBlocked/outProcQ` dalla coda in cui si trovava) e restituito, tramite `freePcb`, alla free list dei PCB, pronto per essere riassegnato a un futuro processo.

## 9 Il test driver ufficiale: `phase1/p1test.c`

`p1test.c` non è codice originale del progetto: è il test driver standard fornito dal corso per verificare `pcb.c` e `asl.c` (compilato solo nel target `phase1` di `CMakeLists.txt`, insieme a `klog.c`, `pcb.c`, `asl.c`). Vale comunque la pena sapere *come è strutturato* e *cosa dimostra*, perché è il modo in cui si osserva la Fase 1 “funzionare” a schermo (gira da sola, senza Nucleus: non ci sono eccezioni né scheduler, è pura esecuzione sequenziale in `main`).

### 9.1 Meccanismo di segnalazione

Due funzioni di supporto, `addokbuf/adderrbuf`, accumulano i messaggi di progresso (rispettivamente di errore) in due buffer statici (`okbuf`, `errbuf`) e li stampano anche a video sul terminale 0 tramite `termprint` (una routine che scrive un carattere alla volta sul registro di comando del device, in polling attivo — non usa `DOIO`, dato che il Nucleus non esiste ancora in questa fase). `adderrbuf` segnala l'errore e poi entra in un **loop infinito** (`while(1);`): non essendoci un vero kernel capace di gestire un `PANIC`, è il modo più semplice per “fermare tutto” non appena qualcosa non torna, così che l'errore resti visibile a video.

### 9.2 Cosa verifica, in ordine

1. `allocPcb/freePcb`: alloca tutti i `MAXPROC` PCB disponibili verificando che nessuna allocazione fallisca, poi verifica che un'allocazione *oltre* il limite restituisca `NULL`; libera 10 PCB per riportarli in free list.
2. **Code di processi** (`insertProcQ/removeProcQ/ headProcQ/outProcQ/emptyProcQ`): costruisce una coda di 10 elementi, verifica che `headProcQ` indichi il primo inserito, toglie un elemento dalla testa e uno dal centro con `outProcQ` (e verifica che togliere un elemento *non presente* restituisca `NULL`), poi svuota la coda con `removeProcQ` controllando che l'ordine di uscita e l'ultimo elemento estratto siano quelli attesi.
3. **Ordinamento per priorità**: inserisce quattro PCB con priorità 20, 10, 5, 10 (l'ultimo, `p2bis`, uguale al secondo) e verifica che `removeProcQ` li restituisca nell'ordine `p1(20)`, `p2(10)`, `p2bis(10)`, `p3(5)`: dimostra sia l'ordinamento per priorità decrescente sia il rispetto del FIFO a parità di priorità (esattamente il comportamento di `insertProcQ` analizzato nella Sezione 5.6).
4. **Albero dei processi** (`insertChild/removeChild/ outChild/emptyChild`): rende 9 PCB figli di uno stesso padre, verifica `outChild` su un figlio in mezzo alla lista e su uno non presente, poi svuota l'albero con `removeChild` controllando che, esaurita la lista, un'ulteriore chiamata restituisca `NULL`.
5. **ASL** (`initASL, insertBlocked, removeBlocked, headBlocked, outBlocked`): blocca processi su diversi semafori, verifica che un descrittore `sem_t` torni davvero alla free list quando la sua coda si svuota (rispolverandolo subito dopo con un nuovo `insertBlocked` sullo stesso indirizzo), e che `outBlocked` rimuova correttamente un processo specifico (anche non in testa) senza poterlo rimuovere una seconda volta.

Se tutti i controlli passano, l'ultimo messaggio stampato è la celebre citazione “*So Long and Thanks for All the Fish*” (Guida galattica per gli autostoppisti): è il segnale, familiare a chi ha seguito corsi analoghi altrove, che l'intera Fase 1 supera i test.

**Nota su `test_asl_manuale.c`.** Nella cartella `phase1/` è presente anche un piccolo programma `test_asl_manuale.c`, scritto per un test manuale rapido della sola ASL (usa `printf` e compila per l'host, *non* fa parte di nessun target CMake): non è codice di produzione né viene eseguito sull'emulatore, ma uno strumento di debug estemporaneo usato durante lo sviluppo per ispezionare a occhio lo stato delle code dei semafori dopo `insertBlocked/removeBlocked/outBlocked`.

## 10 Sistema di build: CMakeLists.txt e config\_machine.json

Anche se non è “codice del kernel”, il sistema di build va conosciuto: determina esattamente quali file finiscono in quale eseguibile e quale configurazione dell’emulatore lo carica — è tipico che un esaminatore chieda “come fai a passare dal sorgente all’esecuzione in Fase 1?”.

### 10.1 CMakeLists.txt — il target phase1

```
1 add_link_options(-nostdlib -T ${URISCV_SRC}/uriscvcore.ldscript -march=rv32imfd)
2 add_compile_options(
3     -ffreestanding -static -nostartfiles -nostdlib
4     -I${URISCV_INC}
5     -ggdb -Wall -O0 -std=gnu99
6     -march=rv32imafd -mabi=ilp32d
7 )
8
9 add_executable(MultiPandOS_phase1 EXCLUDE_FROM_ALL
10     ./klog.c
11     phase1/pcb.c
12     phase1/asl.c
13     phase1/pitest.c
14     ${URISCV_SRC}/crtso.S
15     ${URISCV_SRC}/liburiscv.S
16 )
17
18 add_custom_target(phase1
19     COMMAND ${CMAKE_COMMAND} -E copy
20         ${PROJECT_BINARY_DIR}/MultiPandOS_phase1
21         ${PROJECT_BINARY_DIR}/MultiPandOS
22     COMMAND uriscv-elf2uriscv -k ${PROJECT_BINARY_DIR}/MultiPandOS
23     BYPRODUCTS MultiPandOS.core.uriscv MultiPandOS.stab.uriscv
24     DEPENDS MultiPandOS_phase1
25 )
```

Le opzioni di compilazione globali (`add_compile_options`, `add_link_options`, applicate a tutti e tre i target) riflettono la natura “freestanding” del kernel: `-ffreestanding -nostdlib -nostartfiles` escludono la libc standard e il normale runtime C (non esiste un sistema operativo sottostante che fornisca queste cose: è il progetto stesso a doverle sostituire, es. con `crtso.S` al posto del solito `crt0`), `-march=rv32imafd -mabi=ilp32d` fissano l’architettura RISC-V a 32 bit con estensioni intero/moltiplicazione/ float, `-O0` disabilita le ottimizzazioni (più facile debuggare passo passo nell’emulatore) e il linker script `uriscvcore.ldscript` posiziona sezioni e simboli secondo il formato `.core` atteso dalla macchina virtuale.

Il target `phase1` compila insieme `klog.c`, `pcb.c`, `asl.c` e `pitest.c` (più il runtime di sistema `crtso.S/liburiscv.S`, forniti dall’installazione di uRISCV, non dal progetto) in un unico eseguibile `MultiPandOS_phase1`; il target `custom phase1` lo copia come `MultiPandOS`, lo converte con `uriscv-elf2uriscv -k` nel formato `.core` atteso dall’emulatore (producendo `build/MultiPandOS.core.uriscv` e la relativa tabella dei simboli `.stab.uriscv`, usata dal debugger per mostrare nomi di funzione/variabile invece dei soli indirizzi). I target `phase2` e `phase3` (analizzati nei rispettivi documenti) seguono esattamente la stessa struttura, cambiando solo l’elenco dei sorgenti. `EXCLUDE_FROM_ALL` evita che un semplice `make` (senza specificare il target) provi a compilare tutti e tre insieme, dato che producono lo stesso file di output e si sovrascriverebbero a vicenda.

### 10.2 config\_machine.json — la configurazione dell’emulatore

```
1 {
2     "boot": {
3         "core-file": "build/MultiPandOS.core.uriscv",
4         "load-core-file": true
5     },
6     "bootstrap-rom": "/usr/local/share/uriscv/coreboot.rom.uriscv",
7     "clock-rate": 1,
8     "devices": {
9         "terminal0": {
```

```

10     "enabled": true,
11     "file": "term0.uriscv"
12 }
13 },
14 "execution-rom": "/usr/local/share/uriscv/exec.rom.uriscv",
15 "num-processors": 1,
16 "num-ram-frames": 64,
17 "symbol-table": {
18     "asid": 64,
19     "file": "build/MultiPandOS.stab.uriscv"
20 },
21 "tlb-floor-address": "0x0",
22 "tlb-size": 16
23 }

```

È il file che l'interfaccia grafica di uRISCv/uMPS3 carica per sapere *come* istanziare la macchina virtuale, usato sia per la Fase 1 sia per la Fase 2 (la Fase 3 usa invece `phase3_config_machine.json`, descritto nel documento di Fase 3). Campi rilevanti:

- **boot.core-file**: il file `.core` da caricare all'avvio — esattamente il file prodotto dal target CMake attivo per ultimo (da qui l'avvertenza pratica: bisogna *sempre* ricompilare il target giusto prima di avviare l'emulatore, altrimenti gira il kernel di un'altra fase).
- **bootstrap-rom/execution-rom**: le ROM del firmware dell'emulatore stesso (BIOS), fornite dall'installazione di uRISCv, non dal progetto: sono il "livello 1" (firmware) della gerarchia descritta nell'introduzione.
- **devices.terminal0**: l'unico device configurato in questa fase è un terminale (`term0.uriscv`, un file che l'emulatore usa per registrare/riprodurre l'input-output del terminale). Niente device flash: non serve memoria virtuale né backing store finché non si arriva alla Fase 3.
- **num-ram-frames**: 64 frame di RAM fisica disponibili (sufficienti per il piccolo kernel di Fase 1/2, che non ha bisogno dei 32 frame riservati al Nucleus più i 16 dello Swap Pool della Fase 3 — si confronti con i 256 frame di `phase3_config_machine.json`).
- **symbol-table**: la tabella dei simboli generata insieme al file `.core` (`MultiPandOS.stab.uriscv`), usata dal debugger integrato per mostrare nomi simbolici.
- **tlb-floor-address**: `"0x0"`: in Fase 1/2 non esiste alcuno spazio utente separato da proteggere con la paginazione, quindi il TLB non ha bisogno di un indirizzo di soglia diverso da zero (si confronti con `"0x80000000"` in Fase 3, che corrisponde proprio all'inizio di `kuseg`).